

PROJEKT: DATA-MART- ERSTELLUNG IN SQL

Erarbeitungs-/ Reflexionsphase

B. Sc. Informatik
02.06.2026
Korin Lohmeyer
IU14091946
Dr. Anna Androvitsanea

Inhalt

Zusammenfassung der Implementierung	1
Erklärung Datenbankdesign.....	2
Fachliche Regeln	2
Normalisierung.....	2
Mehrfachbeziehungen	2
Entity-Relationship-Diagramm	3
Dokumentation der SQL-Statements.....	4
Entität Standort	4
Entität Adresse	5
Entität Nutzer	6
Entität Genre	7
Entität Verlag	8
Entität Autor	9
Entität Buch	10
Entität AutorBuch	11
Entität Buchindividuum	12
Entität Buchverfuegbar.....	13
Entität Ausleihe	14
Entität Bewertung.....	15
Entität Versandfirma.....	16
Entität Versand	17
Entität Abholung.....	18
Ausleihe Übersicht View.....	19
Verfügbare Bücher View	20
Buch Bewertung View	21
Datenwörterbuch	22
Entität Standort	22
Entität Adresse	22
Entität Nutzer	23
Entität Genre	23
Entität Verlag	23
Entität Autor	24
Entität Buch	24

Entität Autorbuch	25
Entität Buchindividuum	25
Entität Buchverfügbar.....	26
Entität Ausleihe	26
Entität Bewertung.....	27
Entität Versandfirma.....	28
Entität Versand	28
Entität Abholung.....	29
Tabellenverzeichnis.....	30
Abbildungsverzeichnis	31
Literaturverzeichnis	33

Zusammenfassung der Implementierung

In dem folgenden Dokument wird die Implementierung der Datenbank für die Buchtausch-App beschrieben. Als Datenbankmanagementsystem wurde PostgreSQL verwendet. Zur Entwicklung und Verwaltung der Datenbank kam zusätzlich das Tool DBeaver zum Einsatz. Dieses ermöglicht eine übersichtlichere Darstellung der Tabellen, Beziehungen und Datensätze und erleichtert dadurch die Entwicklung und Analyse der Datenbank. Die Implementierung wäre grundsätzlich auch ausschließlich mit PostgreSQL und der Verwaltungsoberfläche pgAdmin 4 möglich gewesen.

Die Umsetzung erfolgte auf Grundlage des zuvor entwickelten Entity-Relationship-Modells. Zunächst wurde die Datenbank erstellt und ein entsprechender Datenbankbenutzer eingerichtet. Anschließend wurden für alle definierten Entitäten Tabellen angelegt und mit den zugehörigen Attributen ausgestattet. Darüber hinaus wurden Primärschlüssel und Fremdschlüssel definiert, um die Beziehungen und Kardinalitäten des Datenmodells in der Datenbank abzubilden.

Nach der Erstellung der Tabellen wurden diese mit Testdaten befüllt. Hierzu wurden für jede Tabelle mindestens zehn Datensätze mittels INSERT-Statements eingefügt. Die Testdaten dienen der Überprüfung der Datenstruktur sowie der Funktionsfähigkeit der Beziehungen zwischen den einzelnen Tabellen.

Abschließend wurden verschiedene SQL-Abfragen und Views implementiert, um häufig benötigte Informationen übersichtlich darzustellen. Die Views ermöglichen insbesondere die Darstellung verfügbarer Bücher, eine Übersicht überlaufende Ausleihen sowie die Auswertung von Buchbewertungen. Dadurch werden komplexe Zusammenhänge zwischen mehreren Tabellen vereinfacht dargestellt und die Funktionalität der Datenbank zusätzlich überprüft.

Erklärung Datenbankdesign

Fachliche Regeln

Für das Design wurden folgende Fachliche Regeln definiert:

- Ein Buchexemplar kann nicht gleichzeitig an mehrere Nutzer ausgeliehen werden.
- Ein Verfügbarkeitszeitraum kann höchstens einer Ausleihe zugeordnet werden.
- Bewertungen können nur von tatsächlichen Ausleihen getätigt werden.
- Jedes Buchexemplar besitzt genau einen Eigentümer.
- Ein Nutzer kann mehrere Buchexemplare besitzen.
- Ein Buchexemplar kann nur ausgeliehen werden, wenn es Verfügbar ist.

Die Fachliche Regeln werden zu einem großen Teil durch die eingesetzten Kardinalitäten umgesetzt.

Normalisierung

Das Datenbankschema wurde unter Berücksichtigung der ersten, zweiten und dritten Normalisierungsform entworfen (Deifallah, 2026).

Hierbei ist die erste Normalform erfüllt, da nur atomare Werte verwendet werden. Es werden also keine Listen oder ähnliches gespeichert. Eine zweite Normalform ist gegeben, wenn alle nicht-Schlüssel Attribute vollständig vom Schlüssel Attribut abhängen. Dies ist ebenfalls in dem Datenbankschema gegeben. Attribute, welche auch von anderen Faktoren abhängen können, werden dabei in eigene Entitäten gespeichert. Zum Beispiel wird die Entität *versandfirma* aus der Entität *versand* ausgelagert, da die Attribute *name* und *versandkosten* spezifisch für die Entität *versandfirma* sind. Die dritte Normalform ist ebenfalls weitgehend erfüllt. Diese besagt, dass keine transitive Abhängigkeit innerhalb der Attribute einer Entität bestehen dürfen. Das heißt, dass kein nicht Schlüssel Attribut nicht von einem anderen nicht Schlüssel Attribut abhängen darf. Die dritte Normalform wurde hierbei versucht in allen Entitäten zu erreichen. Dadurch werden redundante Daten vermieden aber es entstehen Entitäten, welche wie Attribut Speicher wirken. Hierzu nennen sind die Entitäten *autor*, *verlag* und *genre*. Trotzdem wurde das Schema so gewählt, da entstehende Kardinalitäten so besser umgesetzt werden können.

Zusätzliche zu nennende Design-Entscheidungen sind, dass sich dazu entschieden wurde, allgemeine Informationen zu einem Buch und ein Buchexemplar voneinander zu trennen. Das bewirkt eine bessere Übersicht in der Datenbank und verschlankt die Entität *buch*. Des Weiteren wurde wird der Abhol- oder Versandprozess auch in zwei Entitäten aufgeteilt, da diese fachlich voneinander getrennt sind.

Mehrfachbeziehungen

Durch das Anwenden der zweiten und dritten Normalform, sowie der Vermeidung von redundanten Daten kommt es in dem Datenbankschema zu mehreren Mehrfachbeziehungen. Hierbei stellen Entitäten Beziehungen zwischen mehreren Entitäten her und benötigen hierzu noch zusätzliche Attribute, um diese Verknüpfung zu beschreiben. Diese sind in der folgenden Tabelle dargestellt:

Tabelle 1: Aufführung der Mehrfachbeziehungen.

Entität	Abhängig von	Zusätzliche Attribute
Ausleihe	Buchindividuum, Nutzer, Buchverfuegbar	Startausleihe, Endausleihe
Buchverfuegbar	Buchindividuum, Standort	Startzeit, Endzeit, Post
Bewertung	Ausleihe, Buchindividuum, Nutzer	Bewertung, Rating, Datum
Versand	Ausleihe, Firma, Adresse	Portobezahlt, Sendungsnummer
Abholung	Ausleihe, Adresse	Abholzeitpunkt
Buchindividuum	Buch, Nutzer	Zustand

Entitäten wie *nutzer* oder *buch* habe ich hier nicht aufgeführt, da diese keine fachlichen Informationen zusammenführen.

Entity-Relationship-Diagramm

In einem ER-Diagramm werden alle wichtigen Elemente einer Datenbankstruktur dargestellt und wird eingesetzt, um Datenschemas zu modellieren (Deifallah, 2026). Das hier gezeigte ER-Diagramm wurde von dem Tool DBeaver erzeugt und zeigt die tatsächlich umgesetzte Datenstruktur.

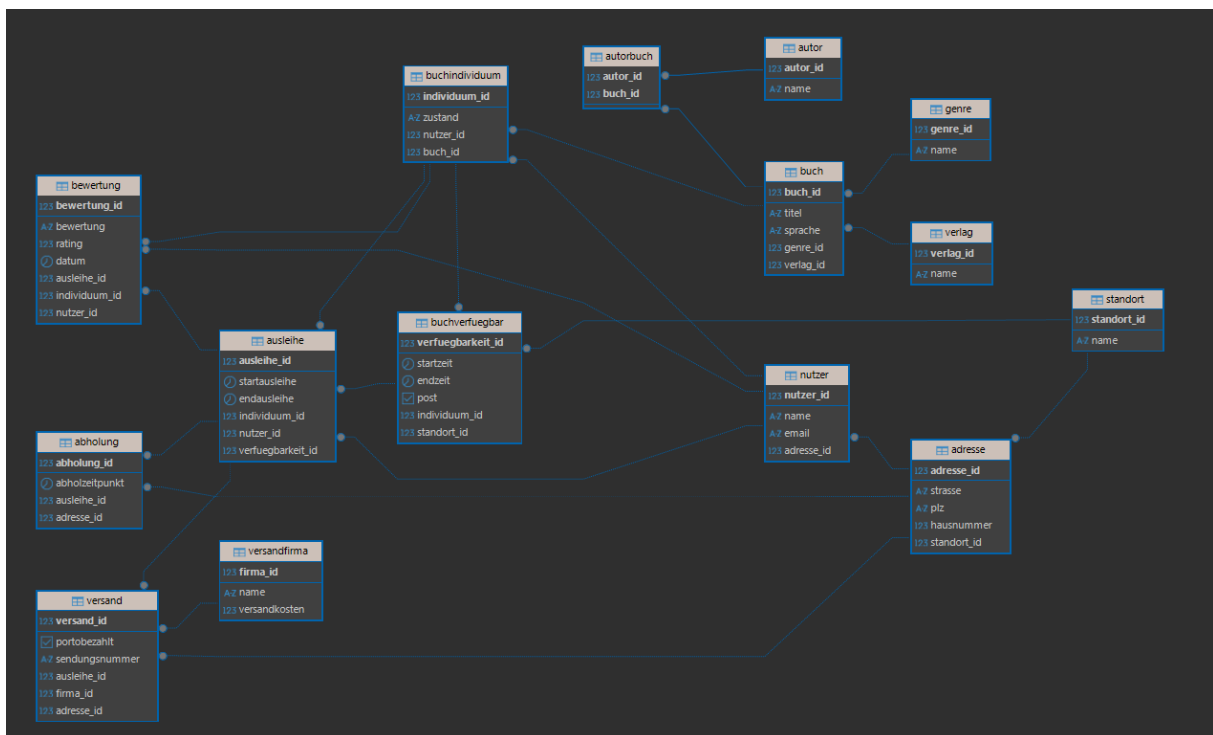


Abbildung 1: Umgesetzte Datenstruktur im ER-Diagramm.

Im Vergleich zu dem ER-Diagramm der Konzeptionsphase fällt auf, dass hier nicht vom Konzept abgewichen wurde. Es wurden lediglich die Entitäten *abholung*, *versand* und *versandfirma* hinzugefügt, um den Anforderungen zu entsprechen.

Dokumentation der SQL-Statements

Entität Standort

```
create table standort(  
    standort_id BIGSERIAL primary key,  
    name varchar(255) not null  
);
```

Abbildung 2: CREATE-Statement Standort.

```
insert into standort (name) values  
('kiel'),  
('hamburg'),  
('flensburg'),  
('lübeck'),  
('bremen'),  
('göttingen'),  
('hannover'),  
('fulda'),  
('kassel'),  
('berlin');
```

Abbildung 3: INSERT-Statement Standort.

select * from standort;

	standort_id	name
1	1	kiel
2	2	hamburg
3	3	flensburg
4	4	lübeck
5	5	bremen
6	6	göttingen
7	7	hannover
8	8	fulda
9	9	kassel
10	10	berlin

Abbildung 4: Ergebnis SELECT-Statement Standort.

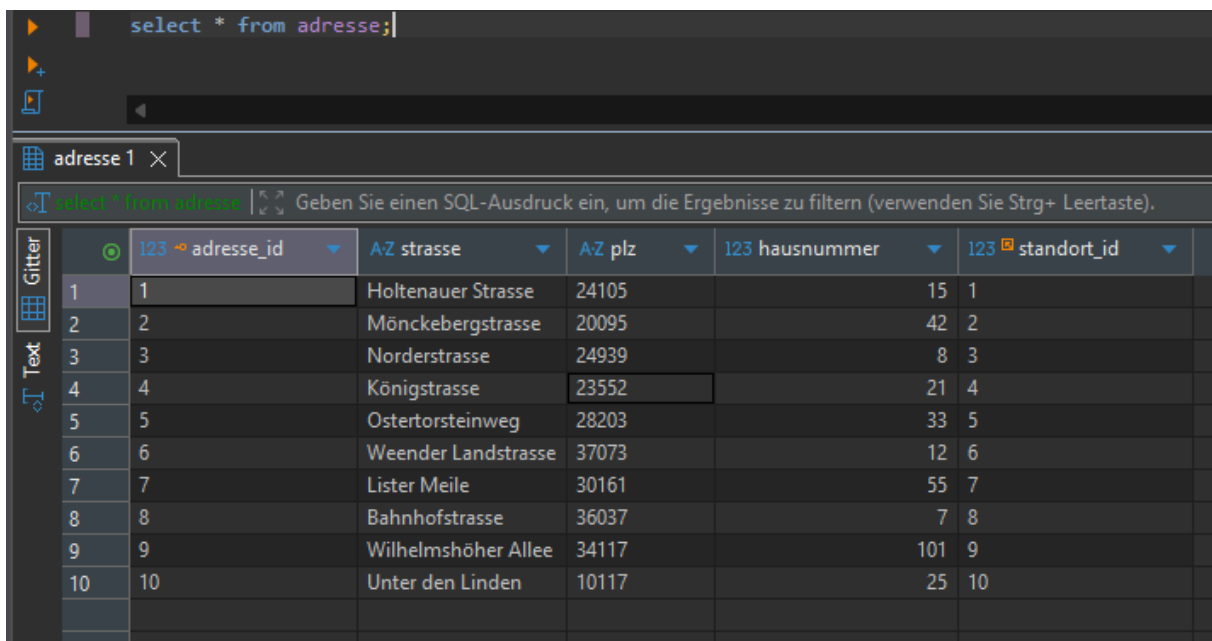
Entität Adresse

```
create table adresse (  
    adresse_id BIGSERIAL PRIMARY KEY,  
    strasse VARCHAR(255) NOT NULL,  
    plz VARCHAR(10) NOT NULL,  
    hausnummer INTEGER NOT NULL,  
    standort_id BIGINT NOT NULL,  
    foreign key(standort_id) references standort(standort_id)  
);
```

Abbildung 5: CREATE-Statement Adresse.

```
insert into adresse (strasse, plz, hausnummer, standort_id) values  
( 'Holtenauer Strasse', '24105', 15, 1),  
( 'Mönckebergstrasse', '20095', 42, 2),  
( 'Norderstrasse', '24939', 8, 3),  
( 'Königstrasse', '23552', 21, 4),  
( 'Ostertorsteinweg', '28203', 33, 5),  
( 'Weender Landstrasse', '37073', 12, 6),  
( 'Lister Meile', '30161', 55, 7),  
( 'Bahnhofstrasse', '36037', 7, 8),  
( 'Wilhelmshöher Allee', '34117', 101, 9),  
( 'Unter den Linden', '10117', 25, 10);
```

Abbildung 6: INSERT-Statement Adresse.



	123	adresse_id	AZ strasse	AZ plz	123 hausnummer	123 standort_id
1	1		Holtenauer Strasse	24105	15	1
2	2		Mönckebergstrasse	20095	42	2
3	3		Norderstrasse	24939	8	3
4	4		Königstrasse	23552	21	4
5	5		Ostertorsteinweg	28203	33	5
6	6		Weender Landstrasse	37073	12	6
7	7		Lister Meile	30161	55	7
8	8		Bahnhofstrasse	36037	7	8
9	9		Wilhelmshöher Allee	34117	101	9
10	10		Unter den Linden	10117	25	10

Abbildung 7: Ergebnis SELECT-Statement Adresse.

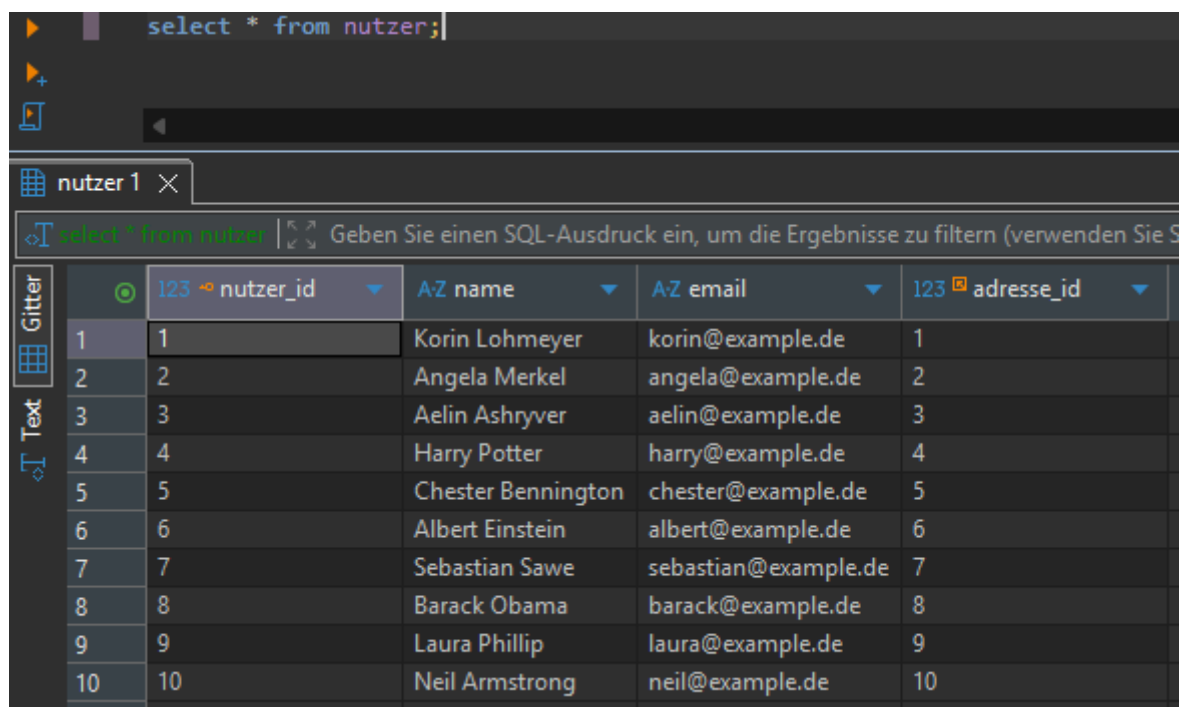
Entität Nutzer

```
create table nutzer(  
    nutzer_id BIGSERIAL primary key,  
    name varchar(255) not null,  
    email varchar(255) not null,  
    adresse_id BIGINT not null,  
    foreign key(adresse_id) references adresse(adresse_id)  
);
```

Abbildung 8: CREATE-Statement Nutzer.

```
insert into nutzer (name, email, adresse_id) values  
( 'Korin Lohmeyer', 'korin@example.de', 1),  
( 'Angela Merkel', 'angela@example.de', 2),  
( 'Aelin Ashryver', 'aelin@example.de', 3),  
( 'Harry Potter', 'harry@example.de', 4),  
( 'Chester Bennington', 'chester@example.de', 5),  
( 'Albert Einstein', 'albert@example.de', 6),  
( 'Sebastian Sawe', 'sebastian@example.de', 7),  
( 'Barack Obama', 'barack@example.de', 8),  
( 'Laura Phillip', 'laura@example.de', 9),  
( 'Neil Armstrong', 'neil@example.de', 10);
```

Abbildung 9: INSERT-Statement Nutzer.



	123 nutzer_id	AZ name	AZ email	123 adresse_id
1	1	Korin Lohmeyer	korin@example.de	1
2	2	Angela Merkel	angela@example.de	2
3	3	Aelin Ashryver	aelin@example.de	3
4	4	Harry Potter	harry@example.de	4
5	5	Chester Bennington	chester@example.de	5
6	6	Albert Einstein	albert@example.de	6
7	7	Sebastian Sawe	sebastian@example.de	7
8	8	Barack Obama	barack@example.de	8
9	9	Laura Phillip	laura@example.de	9
10	10	Neil Armstrong	neil@example.de	10

Abbildung 10: Ergebnis SELECT-Statement Nutzer.

Entität Genre

```
create table genre(  
    genre_id bigserial primary key,  
    name varchar(255) not null  
);
```

Abbildung 11: CREATE-Statement Genre.

```
insert into genre (name) values  
('Fantasy'),  
('Science Fiction'),  
('Thriller'),  
('Krimi'),  
('Roman'),  
('Sachbuch'),  
('Biografie'),  
('Horror'),  
('Historisch'),  
('Jugendbuch');
```

Abbildung 12: INSERT-Statement Genre.

	genre_id	name
1	1	Fantasy
2	2	Science Fiction
3	3	Thriller
4	4	Krimi
5	5	Roman
6	6	Sachbuch
7	7	Biografie
8	8	Horror
9	9	Historisch
10	10	Jugendbuch

Abbildung 13: Ergebnis SELECT-Statement Genre.

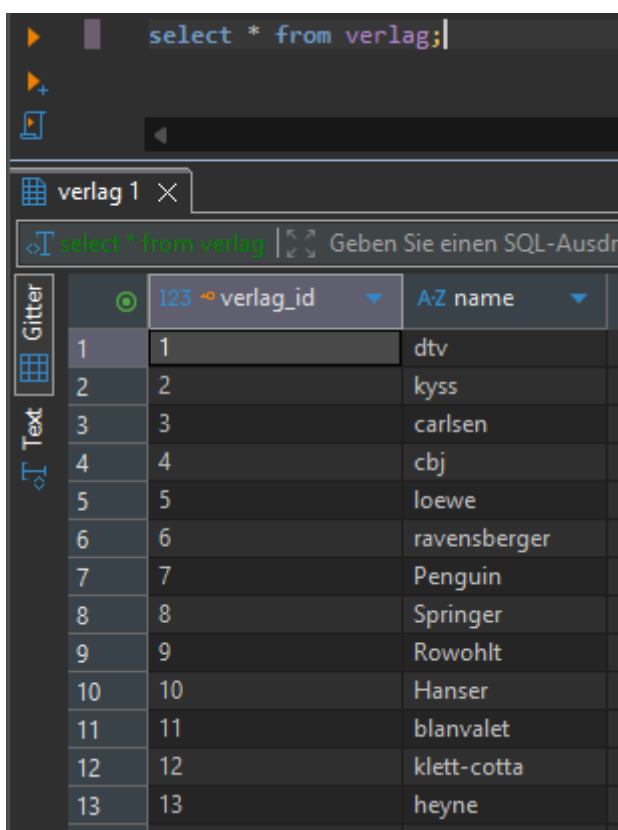
Entität Verlag

```
create table verlag(  
    verlag_id bigserial primary key,  
    name varchar(255) not null  
);
```

Abbildung 14: CREATE-Statement Verlag.

```
insert into verlag (name) values  
('dtv'),  
('kyss'),  
('carlsen'),  
('cbj'),  
('loewe'),  
('ravensberger'),  
('Penguin'),  
('Springer'),  
('Rowohlt'),  
('Hanser'),  
('blanvalet'),  
('klett-cotta'),  
('heyne');
```

Abbildung 15: INSERT-Statement Verlag.



	123 verlag_id	AZ name
1	1	dtv
2	2	kyss
3	3	carlsen
4	4	cbj
5	5	loewe
6	6	ravensberger
7	7	Penguin
8	8	Springer
9	9	Rowohlt
10	10	Hanser
11	11	blanvalet
12	12	klett-cotta
13	13	heyne

Abbildung 16: Ergebnis SELECT-Statement Verlag.

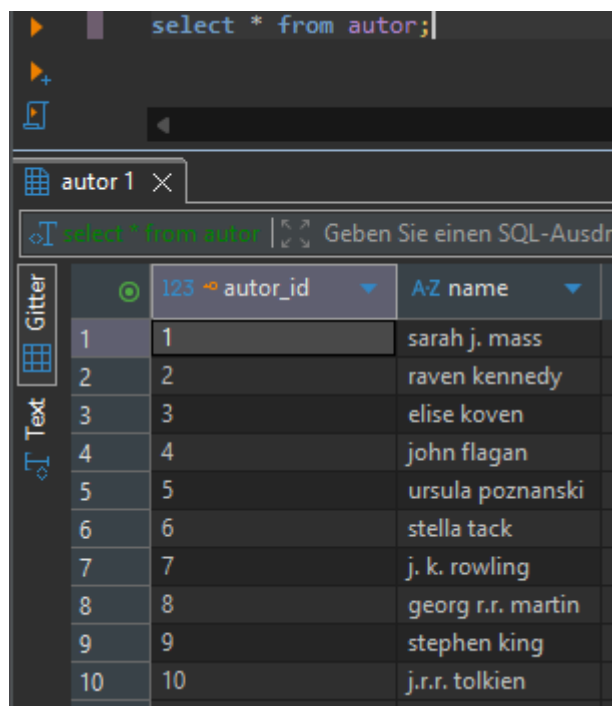
Entität Autor

```
create table autor(  
    autor_id bigserial primary key,  
    name varchar(255) not null  
);
```

Abbildung 17: CREATE-Statement Autor.

```
insert into autor (name) values  
( 'sarah j. mass'),  
( 'raven kennedy'),  
( 'elise koven'),  
( 'john flagan'),  
( 'ursula pozanski'),  
( 'stella tack'),  
( 'j. k. rowling'),  
( 'georg r.r. martin'),  
( 'stephen king'),  
( 'j.r.r. tolkien');
```

Abbildung 18: INSERT-Statement Autor.



	123 autor_id	A-Z name
1	1	sarah j. mass
2	2	raven kennedy
3	3	elise koven
4	4	john flagan
5	5	ursula pozanski
6	6	stella tack
7	7	j. k. rowling
8	8	georg r.r. martin
9	9	stephen king
10	10	j.r.r. tolkien

Abbildung 19: Ergebnis SELECT-Statement Autor.

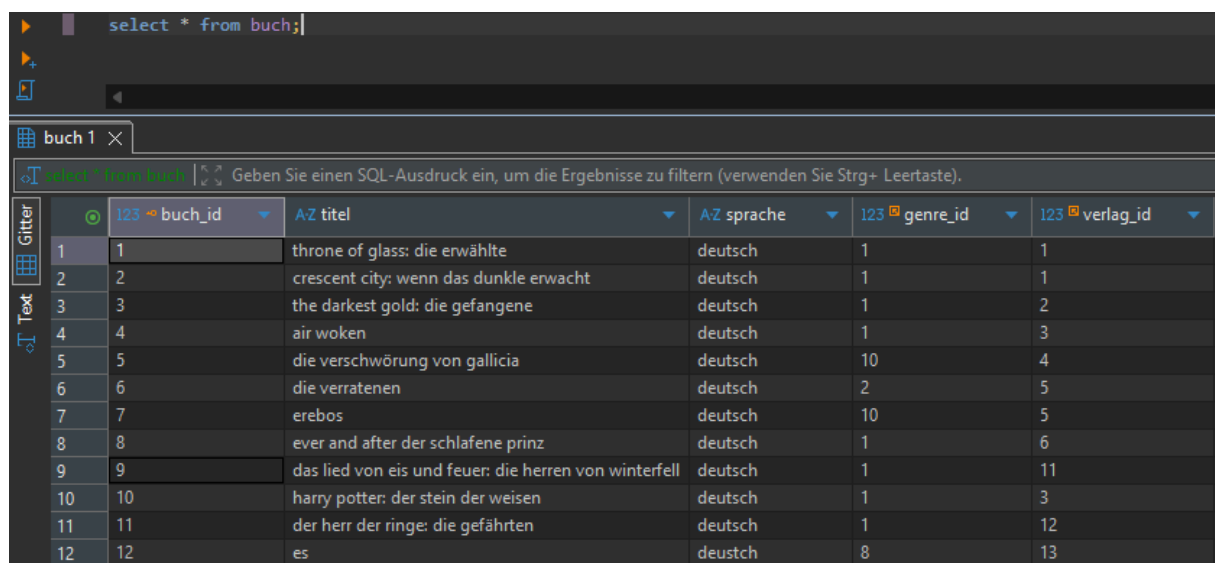
Entität Buch

```
create table buch(  
    buch_id bigserial primary key,  
    titel varchar(255) not null,  
    sprache varchar(255) not null,  
    genre_id bigint not null,  
    foreign key(genre_id) references genre(genre_id),  
    verlag_id bigint not null,  
    foreign key(verlag_id) references verlag(verlag_id)  
);
```

Abbildung 20: CREATE-Statement Buch.

```
insert into buch (titel, sprache, genre_id, verlag_id) values  
( 'throne of glass: die erwählte', 'deutsch', 1, 1),  
( 'crescent city: wenn das dunkle erwacht', 'deutsch', 1, 1 ),  
( 'the darkest gold: die gefangene', 'deutsch', 1, 2),  
( 'air woken', 'deutsch', 1, 3),  
( 'die verschwörung von gallicia', 'deutsch', 10, 4),  
( 'die verratenen', 'deutsch', 2, 5),  
( 'erebos', 'deutsch', 10, 5),  
( 'ever and after der schlafene prinz', 'deutsch', 1, 6),  
( 'das lied von eis und feuer: die herren von winterfell', 'deutsch', 1, 11),  
( 'harry potter: der stein der weisen', 'deutsch', 1, 3),  
( 'der herr der ringe: die gefährten', 'deutsch', 1, 12),  
( 'es', 'deustch', 8, 13);
```

Abbildung 21: INSERT-Statement Buch.



	123 buch_id	AZ titel	AZ sprache	123 genre_id	123 verlag_id
1	1	throne of glass: die erwählte	deutsch	1	1
2	2	crescent city: wenn das dunkle erwacht	deutsch	1	1
3	3	the darkest gold: die gefangene	deutsch	1	2
4	4	air woken	deutsch	1	3
5	5	die verschwörung von gallicia	deutsch	10	4
6	6	die verratenen	deutsch	2	5
7	7	erebos	deutsch	10	5
8	8	ever and after der schlafene prinz	deutsch	1	6
9	9	das lied von eis und feuer: die herren von winterfell	deutsch	1	11
10	10	harry potter: der stein der weisen	deutsch	1	3
11	11	der herr der ringe: die gefährten	deutsch	1	12
12	12	es	deustch	8	13

Abbildung 22: Ergebnis SELECT-Statement Buch.

Entität AutorBuch

```
create table autorbuch(  
    autor_id bigint not null,  
    buch_id bigint not null,  
    primary key (autor_id, buch_id),  
    foreign key (autor_id) references autor (autor_id),  
    foreign key (buch_id) references buch (buch_id)  
);
```

Abbildung 23: CREATE-Statement AutorBuch.

```
insert into autorbuch (autor_id, buch_id) values  
(1, 1),  
(1, 2),  
(2, 3),  
(3, 4),  
(4, 5),  
(5, 6),  
(5, 7),  
(6, 8),  
(8, 9),  
(7, 10),  
(10, 11),  
(9, 12);
```

Abbildung 24: INSERT-Statement AutorBuch.

select * from autorbuch;

	123 autor_id	123 buch_id
1	1	1
2	1	2
3	2	3
4	3	4
5	4	5
6	5	6
7	5	7
8	6	8
9	8	9
10	7	10
11	10	11
12	9	12

Abbildung 25: Ergebnis SELECT-Statement AutorBuch.

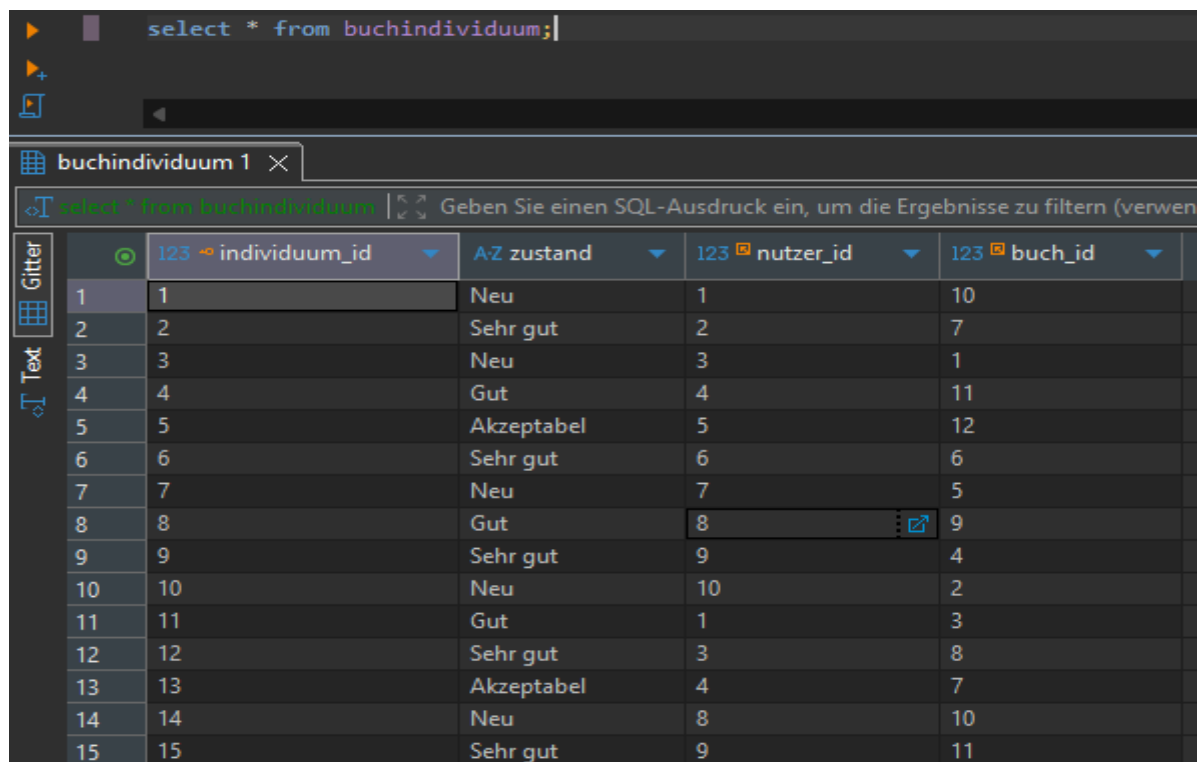
Entität Buchindividuum

```
create table buchindividuum(  
    individuum_id bigserial primary key,  
    zustand varchar(255) not null,  
    nutzer_id bigint not null,  
    foreign key(nutzer_id) references nutzer(nutzer_id),  
    buch_id bigint not null,  
    foreign key(buch_id) references buch(buch_id)  
);
```

Abbildung 26: CREATE-Statement Buchindividuum.

```
insert into buchindividuum (zustand, nutzer_id, buch_id) values  
( 'Neu', 1, 10),  
( 'Sehr gut', 2, 7),  
( 'Neu', 3, 1),  
( 'Gut', 4, 11),  
( 'Akzeptabel', 5, 12),  
( 'Sehr gut', 6, 6),  
( 'Neu', 7, 5),  
( 'Gut', 8, 9),  
( 'Sehr gut', 9, 4),  
( 'Neu', 10, 2),  
( 'Gut', 1, 3),  
( 'Sehr gut', 3, 8),  
( 'Akzeptabel', 4, 7),  
( 'Neu', 8, 10),  
( 'Sehr gut', 9, 11);
```

Abbildung 27: INSERT-Statement Buchindividuum.



```
select * from buchindividuum;
```

	123 individuum_id	AZ zustand	123 nutzer_id	123 buch_id
1	1	Neu	1	10
2	2	Sehr gut	2	7
3	3	Neu	3	1
4	4	Gut	4	11
5	5	Akzeptabel	5	12
6	6	Sehr gut	6	6
7	7	Neu	7	5
8	8	Gut	8	9
9	9	Sehr gut	9	4
10	10	Neu	10	2
11	11	Gut	1	3
12	12	Sehr gut	3	8
13	13	Akzeptabel	4	7
14	14	Neu	8	10
15	15	Sehr gut	9	11

Abbildung 28: Ergebnis SELECT-Statement Buchindividuum.

Entität Buchverfuegbar

```
create table buchverfuegbar(  
    verfuegbarkeit_id bigserial primary key,  
    startzeit timestamp not null,  
    endzeit timestamp not null,  
    post bool not null,  
    individuum_id bigint not null,  
    foreign key(individuum_id) references buchindividuum(individuum_id),  
    standort_id bigint not null,  
    foreign key(standort_id) references standort(standort_id)  
);
```

Abbildung 29: CREATE-Statement Buchverfuegbar.

```
insert into buchverfuegbar (startzeit, endzeit, post, individuum_id, standort_id) values  
( '2026-06-10 10:00:00', '2026-06-20 18:00:00', true, 1, 1),  
( '2026-06-12 09:00:00', '2026-06-22 17:00:00', false, 2, 2),  
( '2026-06-15 12:00:00', '2026-06-25 18:00:00', true, 3, 3),  
( '2026-06-18 14:00:00', '2026-06-28 16:00:00', false, 4, 4),  
( '2026-06-20 10:00:00', '2026-06-30 18:00:00', true, 5, 5),  
( '2026-07-01 09:00:00', '2026-07-10 17:00:00', false, 6, 6),  
( '2026-07-03 11:00:00', '2026-07-13 18:00:00', true, 7, 7),  
( '2026-07-05 10:00:00', '2026-07-15 16:00:00', false, 8, 8),  
( '2026-07-08 13:00:00', '2026-07-18 18:00:00', true, 9, 9),  
( '2026-07-10 09:00:00', '2026-07-20 17:00:00', false, 10, 10),  
( '2026-07-22 10:00:00', '2026-08-01 18:00:00', true, 11, 1),  
( '2026-07-24 09:00:00', '2026-08-03 17:00:00', false, 12, 3),  
( '2026-07-26 12:00:00', '2026-08-05 18:00:00', true, 13, 4),  
( '2026-07-28 14:00:00', '2026-08-07 16:00:00', false, 14, 8),  
( '2026-07-30 10:00:00', '2026-08-09 18:00:00', true, 15, 9);
```

Abbildung 30: INSERT-Statement Buchverfuegbar.

	123 verfuegbarkeit_id	startzeit	endzeit	post	123 individuum_id	123 standort_id
1	1	2026-06-10 10:00:00.000	2026-06-20 18:00:00.000	[v]	1	1
2	2	2026-06-12 09:00:00.000	2026-06-22 17:00:00.000	[]	2	2
3	3	2026-06-15 12:00:00.000	2026-06-25 18:00:00.000	[v]	3	3
4	4	2026-06-18 14:00:00.000	2026-06-28 16:00:00.000	[]	4	4
5	5	2026-06-20 10:00:00.000	2026-06-30 18:00:00.000	[v]	5	5
6	6	2026-07-01 09:00:00.000	2026-07-10 17:00:00.000	[]	6	6
7	7	2026-07-03 11:00:00.000	2026-07-13 18:00:00.000	[v]	7	7
8	8	2026-07-05 10:00:00.000	2026-07-15 16:00:00.000	[]	8	8
9	9	2026-07-08 13:00:00.000	2026-07-18 18:00:00.000	[v]	9	9
10	10	2026-07-10 09:00:00.000	2026-07-20 17:00:00.000	[]	10	10
11	11	2026-07-22 10:00:00.000	2026-08-01 18:00:00.000	[v]	11	1
12	12	2026-07-24 09:00:00.000	2026-08-03 17:00:00.000	[]	12	3
13	13	2026-07-26 12:00:00.000	2026-08-05 18:00:00.000	[v]	13	4
14	14	2026-07-28 14:00:00.000	2026-08-07 16:00:00.000	[]	14	8
15	15	2026-07-30 10:00:00.000	2026-08-09 18:00:00.000	[v]	15	9

Abbildung 31: Ergebnis SELECT-Statement Buchverfuegbar.

Entität Ausleihe

```
create table ausleihe(  
    ausleihe_id bigserial primary key,  
    startausleihe timestamp not null,  
    endausleihe timestamp not null,  
    individuum_id bigint not null,  
    foreign key(individuum_id) references buchindividuum(individuum_id),  
    nutzer_id bigint not null,  
    foreign key(nutzer_id) references nutzer(nutzer_id),  
    verfuegbarkeit_id bigint not null unique,  
    foreign key(verfuegbarkeit_id) references buchverfuegbar(verfuegbarkeit_id)  
);
```

Abbildung 32: CREATE-Statement Ausleihe.

```
insert into ausleihe (startausleihe, endausleihe, individuum_id, nutzer_id, verfuegbarkeit_id)  
values  
( '2026-06-11 10:00:00', '2026-06-18 18:00:00', 1, 2, 1),  
( '2026-06-13 09:00:00', '2026-06-20 17:00:00', 2, 3, 2),  
( '2026-06-16 12:00:00', '2026-06-23 18:00:00', 3, 4, 3),  
( '2026-06-19 14:00:00', '2026-06-26 16:00:00', 4, 5, 4),  
( '2026-06-21 10:00:00', '2026-06-28 18:00:00', 5, 6, 5),  
( '2026-07-02 09:00:00', '2026-07-09 17:00:00', 6, 7, 6),  
( '2026-07-04 11:00:00', '2026-07-11 18:00:00', 7, 8, 7),  
( '2026-07-06 10:00:00', '2026-07-13 16:00:00', 8, 9, 8),  
( '2026-07-09 13:00:00', '2026-07-16 18:00:00', 9, 10, 9),  
( '2026-07-11 09:00:00', '2026-07-18 17:00:00', 10, 1, 10);
```

Abbildung 33: INSERT-Statement Ausleihe.

select * from ausleihe;

	123 ausleihe_id	startausleihe	endausleihe	123 individuum_id	123 nutzer_id	123 verfuegbarkeit_id
1	1	2026-06-11 10:00:00.000	2026-06-18 18:00:00.000	1	2	1
2	2	2026-06-13 09:00:00.000	2026-06-20 17:00:00.000	2	3	2
3	3	2026-06-16 12:00:00.000	2026-06-23 18:00:00.000	3	4	3
4	4	2026-06-19 14:00:00.000	2026-06-26 16:00:00.000	4	5	4
5	5	2026-06-21 10:00:00.000	2026-06-28 18:00:00.000	5	6	5
6	6	2026-07-02 09:00:00.000	2026-07-09 17:00:00.000	6	7	6
7	7	2026-07-04 11:00:00.000	2026-07-11 18:00:00.000	7	8	7
8	8	2026-07-06 10:00:00.000	2026-07-13 16:00:00.000	8	9	8
9	9	2026-07-09 13:00:00.000	2026-07-16 18:00:00.000	9	10	9
10	10	2026-07-11 09:00:00.000	2026-07-18 17:00:00.000	10	1	10

Abbildung 34: Ergebnis SELECT-Statement Ausleihe.

Entität Bewertung

```
create table bewertung(  
    bewertung_id bigserial primary key,  
    bewertung text not null,  
    rating int not null,  
    datum timestamp not null,  
    ausleihe_id bigint not null unique,  
    foreign key(ausleihe_id) references ausleihe(ausleihe_id),  
    individuum_id bigint not null,  
    foreign key(individuum_id) references buchindividuum(individuum_id),  
    nutzer_id bigint not null,  
    foreign key(nutzer_id) references nutzer(nutzer_id)  
);
```

Abbildung 35: CREATE-Statement Bewertung.

```
insert into bewertung (bewertung, rating, datum, ausleihe_id, individuum_id, nutzer_id)  
values  
(  
    'Tolles Buch, schnelle Übergabe.', 5, '2026-06-19 18:00:00', 1, 1, 2),  
    ('Spannende Geschichte, gerne wieder.', 5, '2026-06-21 17:30:00', 2, 2, 3),  
    ('Buch war in sehr gutem Zustand.', 4, '2026-06-24 18:00:00', 3, 3, 4),  
    ('Hat mir sehr gefallen.', 5, '2026-06-27 16:30:00', 4, 4, 5),  
    ('Ließ sich angenehm lesen.', 4, '2026-06-29 18:00:00', 5, 5, 6),  
    ('Interessante Handlung und netter Kontakt.', 5, '2026-07-10 17:00:00', 6, 6, 7),  
    ('Alles problemlos verlaufen.', 5, '2026-07-12 18:00:00', 7, 7, 8),  
    ('Buch entsprach der Beschreibung.', 4, '2026-07-14 16:00:00', 8, 8, 9),  
    ('Sehr empfehlenswert.', 5, '2026-07-17 18:00:00', 9, 9, 10),  
    ('Freundlicher Austausch und tolles Buch.', 5, '2026-07-19 17:00:00', 10, 10, 1);
```

Abbildung 36: INSERT-Statement Bewertung.

select * from bewertung;

	123 bewertung_id	A2 bewertung	123 rating	datum	123 ausleihe_id	123 individuum_id	123 nutzer_id
1	1	Tolles Buch, schnelle Übergabe.	5	2026-06-19 18:00:00.000	1	1	2
2	2	Spannende Geschichte, gerne wieder.	5	2026-06-21 17:30:00.000	2	2	3
3	3	Buch war in sehr gutem Zustand.	4	2026-06-24 18:00:00.000	3	3	4
4	4	Hat mir sehr gefallen.	5	2026-06-27 16:30:00.000	4	4	5
5	5	Ließ sich angenehm lesen.	4	2026-06-29 18:00:00.000	5	5	6
6	6	Interessante Handlung und netter Kontakt.	5	2026-07-10 17:00:00.000	6	6	7
7	7	Alles problemlos verlaufen.	5	2026-07-12 18:00:00.000	7	7	8
8	8	Buch entsprach der Beschreibung.	4	2026-07-14 16:00:00.000	8	8	9
9	9	Sehr empfehlenswert.	5	2026-07-17 18:00:00.000	9	9	10
10	10	Freundlicher Austausch und tolles Buch.	5	2026-07-19 17:00:00.000	10	10	1

Abbildung 37: Ergebnis SELECT-Statement Bewertung.

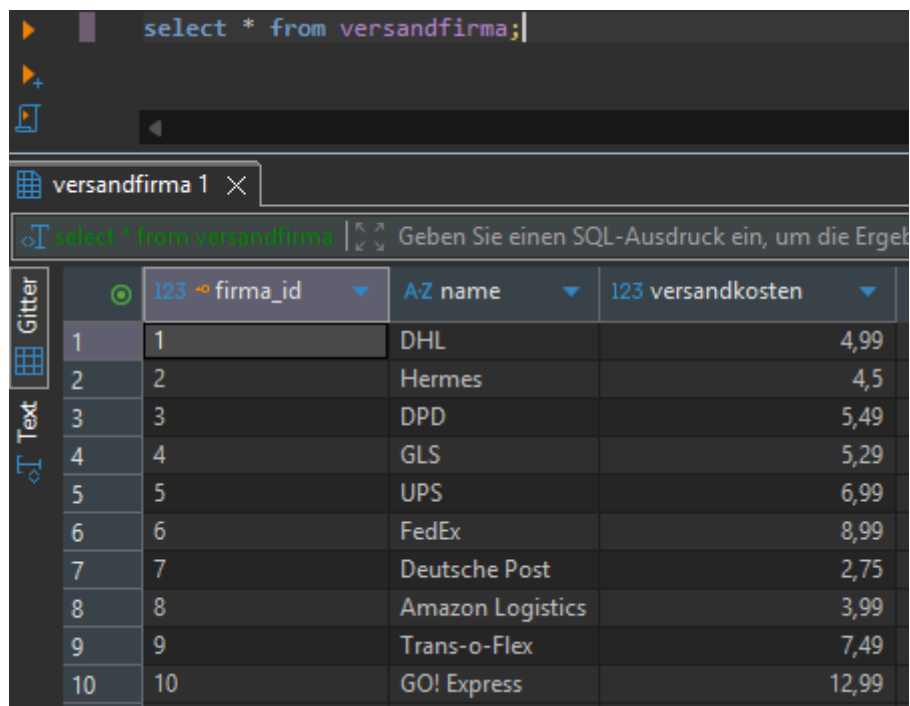
Entität Versandfirma

```
create table versandfirma(  
    firma_id bigserial primary key,  
    name varchar(255) not null,  
    versandkosten numeric(10,2) not null  
);
```

Abbildung 38: CREATE-Statement Versandfirma.

```
insert into versandfirma (name,versandkosten)  
values  
('DHL', 4.99),  
('Hermes', 4.50),  
('DPD', 5.49),  
('GLS', 5.29),  
('UPS', 6.99),  
('FedEx', 8.99),  
('Deutsche Post', 2.75),  
('Amazon Logistics', 3.99),  
('Trans-o-Flex', 7.49),  
('GO! Express', 12.99);
```

Abbildung 39: INSERT-Statement Versandfirma.



	123 firma_id	A-Z name	123 versandkosten
1	1	DHL	4,99
2	2	Hermes	4,5
3	3	DPD	5,49
4	4	GLS	5,29
5	5	UPS	6,99
6	6	FedEx	8,99
7	7	Deutsche Post	2,75
8	8	Amazon Logistics	3,99
9	9	Trans-o-Flex	7,49
10	10	GO! Express	12,99

Abbildung 40: Ergebnis SELECT-Statement Versandfirma.

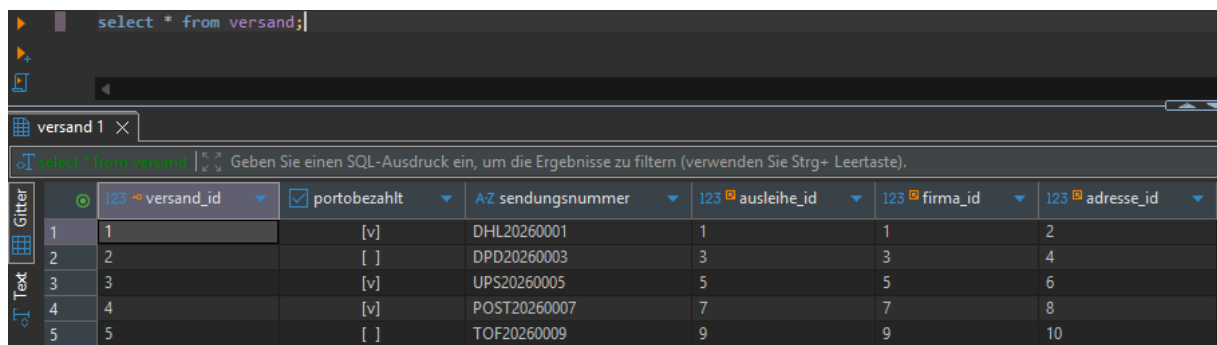
Entität Versand

```
create table versand(  
    versand_id bigserial primary key,  
    portobezahlt bool not null,  
    sendungsnummer varchar(255) not null,  
    ausleihe_id bigint not null,  
    foreign key(ausleihe_id) references ausleihe(ausleihe_id),  
    firma_id bigint not null,  
    foreign key(firma_id) references versandfirma(firma_id),  
    adresse_id bigint not null,  
    foreign key(adresse_id) references adresse(adresse_id)  
);
```

Abbildung 41: CREATE-Statement Versand.

```
insert into versand ( portobezahlt, sendungsnummer, ausleihe_id, firma_id, adresse_id)  
values  
(true, 'DHL20260001', 1, 1, 2),  
(false, 'DPD20260003', 3, 3, 4),  
(true, 'UPS20260005', 5, 5, 6),  
(true, 'POST20260007', 7, 7, 8),  
(false, 'TOF20260009', 9, 9, 10);
```

Abbildung 42: INSERT-Statement Versand.



	123 → versand_id	portobezahlt	A-Z sendungsnummer	123 → ausleihe_id	123 → firma_id	123 → adresse_id
1	1	[v]	DHL20260001	1	1	2
2	2	[]	DPD20260003	3	3	4
3	3	[v]	UPS20260005	5	5	6
4	4	[v]	POST20260007	7	7	8
5	5	[]	TOF20260009	9	9	10

Abbildung 43: Ergebnis SELECT-Statement Versand.

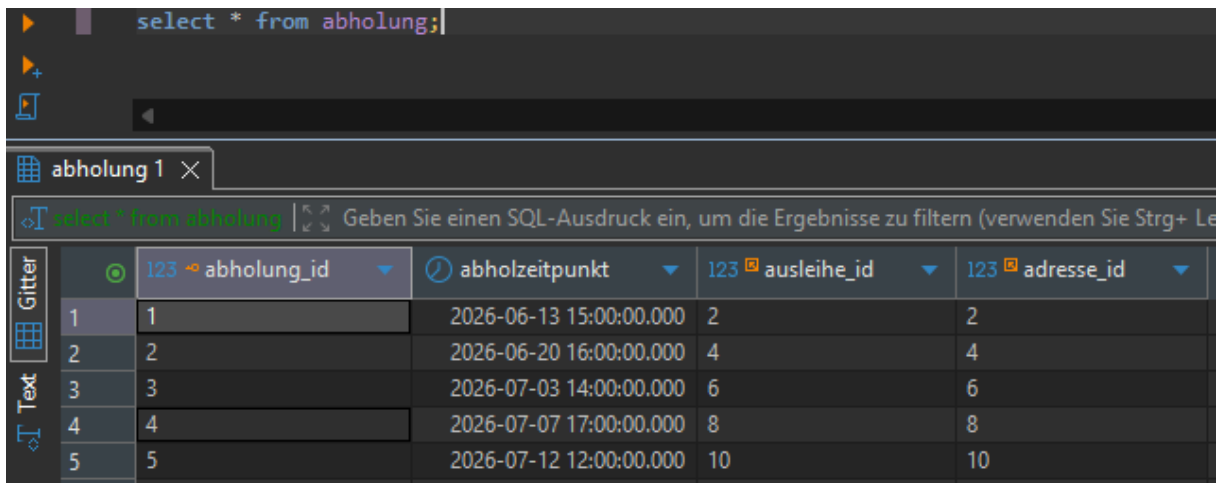
Entität Abholung

```
create table abholung(  
    abholung_id bigserial primary key,  
    abholzeitpunkt timestamp not null,  
    ausleihe_id bigint not null,  
    foreign key(ausleihe_id) references ausleihe(ausleihe_id),  
    adresse_id bigint not null,  
    foreign key(adresse_id) references adresse(adresse_id)  
);
```

Abbildung 44: CREATE-Statement Abholung.

```
insert into abholung ( abholzeitpunkt, ausleihe_id, adresse_id)  
values  
('2026-06-13 15:00:00', 2, 2),  
('2026-06-20 16:00:00', 4, 4),  
('2026-07-03 14:00:00', 6, 6),  
('2026-07-07 17:00:00', 8, 8),  
('2026-07-12 12:00:00', 10, 10);
```

Abbildung 45: INSERT-Statement Abholung.



	123 abholung_id	abholzeitpunkt	123 ausleihe_id	123 adresse_id
1	1	2026-06-13 15:00:00.000	2	2
2	2	2026-06-20 16:00:00.000	4	4
3	3	2026-07-03 14:00:00.000	6	6
4	4	2026-07-07 17:00:00.000	8	8
5	5	2026-07-12 12:00:00.000	10	10

Abbildung 46: Ergebnis SELECT-Statement Abholung.

Ausleihe Übersicht View

```
create view ausleihe_uebersicht as
select
  a.ausleihe_id,
  n.name as ausleiher,
  n2.name as besitzer,
  b2.titel,
  a.startausleihe,
  a.endausleihe
from ausleihe a
join nutzer n on a.nutzer_id = n.nutzer_id
join buchindividuum b on a.individuum_id = b.individuum_id
join nutzer n2 on b.nutzer_id = n2.nutzer_id
join buch b2 on b.buch_id = b2.buch_id;
```

Abbildung 47: CREATE-Statement für Ausleihen-Übersicht View.

select * from ausleihe_uebersicht;

	ausleihe_id	AZ ausleiher	AZ besitzer	AZ titel	startausleihe	endausleihe
1	1	Angela Merkel	Korin Lohmeyer	harry potter: der stein der weisen	2026-06-11 10:00:00.000	2026-06-18 18:00:00.000
2	2	Aelin Ashryver	Angela Merkel	erebos	2026-06-13 09:00:00.000	2026-06-20 17:00:00.000
3	3	Harry Potter	Aelin Ashryver	throne of glass: die erwählte	2026-06-16 12:00:00.000	2026-06-23 18:00:00.000
4	4	Chester Bennington	Harry Potter	der herr der ringe: die gefährten	2026-06-19 14:00:00.000	2026-06-26 16:00:00.000
5	5	Albert Einstein	Chester Bennington	es	2026-06-21 10:00:00.000	2026-06-28 18:00:00.000
6	6	Sebastian Sawe	Albert Einstein	die verratenen	2026-07-02 09:00:00.000	2026-07-09 17:00:00.000
7	7	Barack Obama	Sebastian Sawe	die verschwörung von gallicia	2026-07-04 11:00:00.000	2026-07-11 18:00:00.000
8	8	Laura Phillip	Barack Obama	das lied von eis und feuer: die herren von winterfell	2026-07-06 10:00:00.000	2026-07-13 16:00:00.000
9	9	Neil Armstrong	Laura Phillip	air woken	2026-07-09 13:00:00.000	2026-07-16 18:00:00.000
10	10	Korin Lohmeyer	Neil Armstrong	crescent city: wenn das dunkle erwacht	2026-07-11 09:00:00.000	2026-07-18 17:00:00.000

Abbildung 48: Ergebnis SELECT-Statement Ausleihe-Übersicht View.

Verfügbare Bücher View

```
create view available_books_view as
select
  b.buch_id,
  a.titel,
  b.zustand,
  n.name,
  b2.startzeit,
  b2.endzeit,
  s.name as standort
from buchindividuum b
join buch a on b.buch_id = a.buch_id
join nutzer n on b.nutzer_id = n.nutzer_id
join adresse a2 on n.adresse_id = a2.adresse_id
join standort s on a2.standort_id = s.standort_id
join buchverfuegbar b2 on b.individuum_id = b2.individuum_id;
```

Abbildung 49: CREATE-Statement für verfügbare Bücher View.

select * from available_books_view;

available_books_view 1 X

select * from available_books_view | Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste).

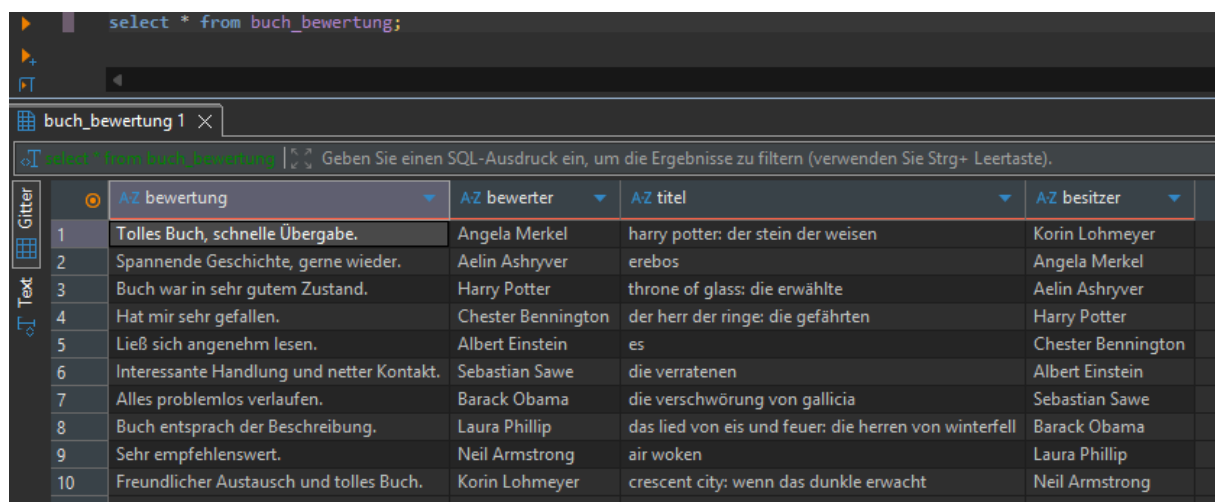
	123 buch_id	AZ titel	AZ zustand	AZ name	startzeit	endzeit	AZ standort
1	10	harry potter: der stein der weisen	Neu	Korin Lohmeyer	2026-06-10 10:00:00.000	2026-06-20 18:00:00.000	kiel
2	7	erebos	Sehr gut	Angela Merkel	2026-06-12 09:00:00.000	2026-06-22 17:00:00.000	hamburg
3	1	throne of glass: die erwählte	Neu	Aelin Ashryver	2026-06-15 12:00:00.000	2026-06-25 18:00:00.000	flensburg
4	11	der herr der ringe: die gefährten	Gut	Harry Potter	2026-06-18 14:00:00.000	2026-06-28 16:00:00.000	lübeck
5	12	es	Akzeptabel	Chester Bennington	2026-06-20 10:00:00.000	2026-06-30 18:00:00.000	bremen
6	6	die verratenen	Sehr gut	Albert Einstein	2026-07-01 09:00:00.000	2026-07-10 17:00:00.000	göttingen
7	5	die verschwörung von gallicia	Neu	Sebastian Sawe	2026-07-03 11:00:00.000	2026-07-13 18:00:00.000	hannover
8	9	das lied von eis und feuer: die herren von winterfell	Gut	Barack Obama	2026-07-05 10:00:00.000	2026-07-15 16:00:00.000	fulda
9	4	air woken	Sehr gut	Laura Phillip	2026-07-08 13:00:00.000	2026-07-18 18:00:00.000	kassel
10	2	crescent city: wenn das dunkle erwacht	Neu	Neil Armstrong	2026-07-10 09:00:00.000	2026-07-20 17:00:00.000	berlin
11	3	the darkest gold: die gefangene	Gut	Korin Lohmeyer	2026-07-22 10:00:00.000	2026-08-01 18:00:00.000	kiel
12	8	ever and after der schlafene prinz	Sehr gut	Aelin Ashryver	2026-07-24 09:00:00.000	2026-08-03 17:00:00.000	flensburg
13	7	erebos	Akzeptabel	Harry Potter	2026-07-26 12:00:00.000	2026-08-05 18:00:00.000	lübeck
14	10	harry potter: der stein der weisen	Neu	Barack Obama	2026-07-28 14:00:00.000	2026-08-07 16:00:00.000	fulda
15	11	der herr der ringe: die gefährten	Sehr gut	Laura Phillip	2026-07-30 10:00:00.000	2026-08-09 18:00:00.000	kassel

Abbildung 50: Ergebnis SELECT-Statement verfügbare Bücher View.

Buch Bewertung View

```
create view buch_bewertung as
select
  b.bewertung,
  n.name as bewerter,
  b3.titel,
  n2.name as besitzer
from bewertung b
join nutzer n on b.nutzer_id = n.nutzer_id
join buchindividuum b2 on b.individuum_id = b2.individuum_id
join buch b3 on b2.buch_id = b3.buch_id
join nutzer n2 on b2.nutzer_id = n2.nutzer_id;
```

Abbildung 51: CREATE-Statement Buch Bewertung View.



	AZ bewertung	AZ bewerter	AZ titel	AZ besitzer
1	Tolles Buch, schnelle Übergabe.	Angela Merkel	harry potter: der stein der weisen	Korin Lohmeyer
2	Spannende Geschichte, gerne wieder.	Aelin Ashryver	erebos	Angela Merkel
3	Buch war in sehr gutem Zustand.	Harry Potter	throne of glass: die erwählte	Aelin Ashryver
4	Hat mir sehr gefallen.	Chester Bennington	der herr der ringe: die gefährten	Harry Potter
5	Ließ sich angenehm lesen.	Albert Einstein	es	Chester Bennington
6	Interessante Handlung und netter Kontakt.	Sebastian Sawe	die verratenen	Albert Einstein
7	Alles problemlos verlaufen.	Barack Obama	die verschwörung von gallicia	Sebastian Sawe
8	Buch entsprach der Beschreibung.	Laura Phillip	das lied von eis und feuer: die herren von winterfell	Barack Obama
9	Sehr empfehlenswert.	Neil Armstrong	air woken	Laura Phillip
10	Freundlicher Austausch und tolles Buch.	Korin Lohmeyer	crescent city: wenn das dunkle erwacht	Neil Armstrong

Abbildung 52: Ergebnis SELECT-Statement Buch Bewertung View.

Datenwörterbuch

Entität Standort

Tabelle 2: Datenwörterbuch Standort.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
standort_id	Eindeutige ID für jeden Eintrag	BIGSERIAL	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
name	Name des Standortes/Stadt	varchar(255)	Zeichenkette mit maximal 255 Zeichen.

Entität Adresse

Tabelle 3: Datenwörterbuch Adresse.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
adresse_id	Eindeutige ID für jeden Eintrag	BIGSERIAL	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
strasse	Name der Straße	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
plz	Postleitzahl	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
hausnummer	Hasunummer der Adresse	integer	Datentyp für eine ganzzahlige Zahl.
standort_id	Eindeutige ID für den Standort der Straße. Referenziert auf die Tabelle Standort.	bigint	Datentyp für große ganzzahlige Zahlen.

Entität Nutzer

Tabelle 4: Datenwörterbuch Nutzer.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
nutzer_id	Eindeutige ID für jeden Eintrag.	BIGSERIAL	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
name	Name des Nutzers.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
email	E-Mail-Adresse des Nutzers	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
adresse_id	Eindeutige ID für die Adresse des Nutzers. Referenziert auf die Tabelle Adresse.	bigint	Datentyp für große ganzzahlige Zahlen.

Entität Genre

Tabelle 5: Datenwörterbuch Genre.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
genre_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
name	Name des Genres.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.

Entität Verlag

Tabelle 6: Datenwörterbuch Verlag.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
verlag_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
name	Name des Verlages	varchar(255)	Zeichenkette mit maximal 255 Zeichen.

Entität Autor

Tabelle 7: Datenwörterbuch Autor.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
autor_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
name	Name des Autors.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.

Entität Buch

Tabelle 8: Datenwörterbuch Buch.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
buch_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
titel	Titel des jeweiligen Buches.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
sprache	Sprache, in der das Buch verfasst ist.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
genre_id	Eindeutige ID für das Genre, dem das Buch zuzuordnen ist. Referenziert auf die Tabelle Genre.	bigint	Datentyp für große ganzzahlige Zahlen.
verlag_id	Eindeutige ID für den Verlag, wo das Buch veröffentlicht wurde. Referenziert auf die Tabelle Verlag.	bigint	Datentyp für große ganzzahlige Zahlen.

Entität Autorbuch

Tabelle 9: Datenwörterbuch AutorBuch.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
autor_id	Eindeutige ID für den Autor. Referenziert auf die Tabelle Autor.	bigint	Datentyp für große ganzzahlige Zahlen.
buch_id	Eindeutige ID für das Buch. Referenziert auf die Tabelle Buch.	bigint	Datentyp für große ganzzahlige Zahlen.

Bei dieser Tabelle handelt es sich um eine Beziehungstabelle. Sie wird genutzt, um Autoren und Bücher zusammen zu bringen, da ein Buch von mehreren Autoren geschrieben sein kann aber auch ein Autor mehrere Bücher geschrieben haben kann. Der primäre Schlüssel der Einträge setzt sich aus den beiden fremd Schlüsseln zusammen.

Entität Buchindividuum

Tabelle 10: Datenwörterbuch Buchindividuum.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
individuum_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
zustand	Zustand des einzelnen Exemplars	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
nutzer_id	Eindeutige ID für den Nutzer, welcher das Exemplar besitzt. Referenziert auf die Tabelle Nutzer.	bigint	Datentyp für große ganzzahlige Zahlen.
buch_id	Eindeutige ID für Buch. Beschreibt von welchem Buch das Exemplar ist. Referenziert auf die Tabelle Buch.	bigint	Datentyp für große ganzzahlige Zahlen.

Entität Buchverfügbar

Tabelle 11: Datenwörterbuch Buchverfügbar.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
verfuegbarkeit_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
startzeit	Ab dieser Zeit ist das Buch verfügbar und kann ausgeliehen werden.	timestamp	Zeitpunkt mit Datum und Uhrzeit.
endzeit	Bis zu diesem Zeitpunkt ist das Buch verfügbar.	timestamp	Zeitpunkt mit Datum und Uhrzeit.
post	Option, ob das Buch Versand werden kann.	bool	Wahrheitswert, also true oder false.
individuum_id	Eindeutige ID für das Exemplar. Referenziert auf die Tabelle Buchindividuum.	bigint	Datentyp für große ganzzahlige Zahlen.
standort_id	Eindeutige ID für den Standort. Also wo ist das Exemplar verfügbar. Referenziert auf die Tabelle Standort.	bigint	Datentyp für große ganzzahlige Zahlen.

Entität Ausleihe

Tabelle 12: Datenwörterbuch Ausleihe.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
ausleihe_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
startausleihe	Startzeit der Ausleihe	timestamp	Zeitpunkt mit Datum und Uhrzeit.
endausleihe	Endzeitpunkt der Ausleihe.	timestamp	Zeitpunkt mit Datum und Uhrzeit.
individuum_id	Eindeutige ID für das ausgeliehene Exemplar. Referenziert auf die Tabelle Buchindividuum.	bigint	Datentyp für große ganzzahlige Zahlen.

nutzer_id	Eindeutige ID für den Nutzer, welcher das Buch ausleiht. Referenziert auf die Tabelle Nutzer.	bigint	Datentyp für große ganzzahlige Zahlen.
verfuegbarkeit_id	Eindeutige ID für die Verfügbarkeit eines Exemplars. Referenziert auf die Tabelle Buchverfuegbar.	bigint	Datentyp für große ganzzahlige Zahlen.

Eine Besonderheit in dieser Entität ist die Nutzung vom *unique* in dem Kontext des Referenzierens auf die Tabelle *buchverfuegbar*. Dies führt dazu, dass ein Verfügbarkeitszeitraum zu exakt einer oder keiner Ausleihe führt.

Entität Bewertung

Tabelle 13: Datenwörterbuch Bewertung.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
bewertung_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
bewertung	Bewertender Text zu einem Buchexemplar.	text	Potenzielle lange Bewertungen. Deswegen text als Datentyp.
rating	Bewertung eines Buches von 1-10.	int	Ganzzahlige Zahlen
datum	Zeitpunkt der Bewertung.	timestamp	Zeitpunkt mit Datum und Uhrzeit.
ausleihe_id	Eindeutige ID für die Ausleihe. Referenziert auf die Tabelle Ausleihe.	bigint	Datentyp für große ganzzahlige Zahlen.
individuum_id	Eindeutige ID für das Buch Exemplar. Referenziert die Tabelle Buchindividuum.	bigint	Datentyp für große ganzzahlige Zahlen.
nutzer_id	Eindeutige ID für den Nutzer, der den Kommentar verfasst hat. Referenziert die Tabelle Nutzer	bigint	Datentyp für große ganzzahlige Zahlen.

Die hier vorgegebene Struktur ermöglicht nur einen Kommentar pro Ausleihe. Des Weiteren ist es nur möglich einen Kommentar zu verfassen, wenn eine Ausleihe stattgefunden hat.

Entität Versandfirma

Tabelle 14: Datenwörterbuch Versandfirma.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
firma_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
name	Name des Autors.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
versandkosten	Kosten für den Versand von Büchern.	numeric(10,2)	Gleitkommazahlen mit zwei Nachkommastellen.

Entität Versand

Tabelle 15: Datenwörterbuch Versand.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
versand_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
portobezahlt	Wahrheitswert, ob der Versand schon bezahlt wurde.	bool	Wahrheitswert: true/false
sendungsnummer	Sendungsnummer für Statusübersicht des Versands.	varchar(255)	Zeichenkette mit maximal 255 Zeichen.
ausleihe_id	Eindeutige ID für die Ausleihe. Referenziert auf die Tabelle Ausleihe.	bigint	Datentyp für große ganzzahlige Zahlen.
firma_id	Eindeutige ID für die Versandfirma. Also DHL etc.	bigint	Datentyp für große ganzzahlige Zahlen.
adresse_id	Eindeutige ID für die Adresse, wohin das Buch Versand wird. Referenziert die Tabelle Adresse.	bigint	Datentyp für große ganzzahlige Zahlen.

Entität Abholung

Tabelle 16: Datenwörterbuch Abholung.

Name	Beschreibung	Datentyp	Beschreibung Datentyp
abholung_id	Eindeutige ID für jeden Eintrag.	bigserial	Wird automatisch von der Datenbank vergeben, wenn neuer Eintrag hinzugefügt wird.
abholzeitpunkt	Zeitpunkt, an dem das Buch abgeholt wird.	timestamp	Zeitpunkt mit Datum und Uhrzeit.
ausleihe_id	Eindeutige ID für die Ausleihe. Referenziert auf die Tabelle Ausleihe.	bigint	Datentyp für große ganzzahlige Zahlen.
adresse_id	Eindeutige ID für die Adresse, wo das Buch abgeholt werden soll. Referenziert die Tabelle Adresse.	bigint	Datentyp für große ganzzahlige Zahlen.

Tabellenverzeichnis

Tabelle 1: Aufführung der Mehrfachbeziehungen.	3
Tabelle 2: Datenwörterbuch Standort.	22
Tabelle 3: Datenwörterbuch Adresse.	22
Tabelle 4: Datenwörterbuch Nutzer.	23
Tabelle 5: Datenwörterbuch Genre.	23
Tabelle 6: Datenwörterbuch Verlag.	23
Tabelle 7: Datenwörterbuch Autor.	24
Tabelle 8: Datenwörterbuch Buch.	24
Tabelle 9: Datenwörterbuch AutorBuch.	25
Tabelle 10: Datenwörterbuch Buchindividuum.	25
Tabelle 11: Datenwörterbuch Buchverfuegbar.	26
Tabelle 12: Datenwörterbuch Ausleihe.	26
Tabelle 13: Datenwörterbuch Bewertung.	27
Tabelle 14: Datenwörterbuch Versandfirma.	28
Tabelle 15: Datenwörterbuch Versand.	28
Tabelle 16: Datenwörterbuch Abholung.	29

Abbildungsverzeichnis

Abbildung 1: Umgesetzte Datenstruktur im ER-Diagramm.....	3
Abbildung 2: CREATE-Statement Standort.	4
Abbildung 3: INSERT-Statement Standort.	4
Abbildung 4: Ergebnis SELECT-Statement Standort.....	4
Abbildung 5: CREATE-Statement Adresse.....	5
Abbildung 6: INSERT-Statement Adresse.....	5
Abbildung 7: Ergebnis SELECT-Statement Adresse.	5
Abbildung 8: CREATE-Statement Nutzer.	6
Abbildung 9: INSERT-Statement Nutzer.	6
Abbildung 10: Ergebnis SELECT-Statement Nutzer.	6
Abbildung 11: CREATE-Statement Genre.....	7
Abbildung 12: INSERT-Statement Genre.	7
Abbildung 13: Ergebnis SELECT-Statement Genre.....	7
Abbildung 14: CREATE-Statement Verlag.	8
Abbildung 15: INSERT-Statement Verlag.	8
Abbildung 16: Ergebnis SELECT-Statement Verlag.....	8
Abbildung 17: CREATE-Statement Autor.....	9
Abbildung 18: INSERT-Statement Autor.....	9
Abbildung 19: Ergebnis SELECT-Statement Autor.	9
Abbildung 20: CREATE-Statement Buch.	10
Abbildung 21: INSERT-Statement Buch.	10
Abbildung 22: Ergebnis SELECT-Statement Buch.....	10
Abbildung 23: CREATE-Statement AutorBuch.	11
Abbildung 24: INSERT-Statement AutotBuch.	11
Abbildung 25: Ergebnis SELECT-Statement AutorBuch.	11
Abbildung 26: CREATE-Statement Buchindividuum.	12
Abbildung 27: INSERT-Statement Buchindividuum.	12
Abbildung 28: Ergebnis SELECT-Statement Buchindividuum.	12
Abbildung 29: CREATE-Statement Buchverfuegbar.	13
Abbildung 30: INSERT-Statement Buchverfuegbar.	13
Abbildung 31: Ergebnis SELECT-Statement Buchverfuegbar.	13
Abbildung 32: CREATE-Statement Ausleihe.....	14
Abbildung 33: INSERT-Statement Ausleihe.....	14
Abbildung 34: Ergebnis SELECT-Statement Ausleihe.....	14
Abbildung 35: CREATE-Statement Bewertung.....	15
Abbildung 36: INSERT-Statement Bewertung.....	15
Abbildung 37: Ergebnis SELECT-Statement Bewertung.....	15
Abbildung 38: CREATE-Statement Versandfirma.....	16
Abbildung 39: INSERT-Statement Versandfirma.....	16
Abbildung 40: Ergebnis SELECT-Statement Versandfirma.....	16
Abbildung 41: CREATE-Statement Versand.	17
Abbildung 42: INSERT-Statement Versand.	17
Abbildung 43: Ergebnis SELECT-Statement Versand.....	17

Abbildung 44: CREATE-Statement Abholung.....	18
Abbildung 45: INSERT-Statement Abholung.....	18
Abbildung 46: Ergebnis SELECT-Statement Abholung.	18
Abbildung 47: CREATE-Statement für Ausleihen-Übersicht View.....	19
Abbildung 48: Ergebnis SELECT-Statement Ausleihe-Übersicht View.....	19
Abbildung 49: CREATE-Statement für verfügbare Bücher View.	20
Abbildung 50: Ergebnis SELECT-Statement verfügbare Bücher View.	20
Abbildung 51: CREATE-Statement Buch Bewertung View.	21
Abbildung 52: Ergebnis SELECT-Statement Buch Bewertung View.	21

Literaturverzeichnis

Deifallah, D. (2026). *Datenmodellierung und Datenbanksysteme*. Erfurt: IU Internationale Hochschule.

Normalisierung (Datenbank). (11. Juni 2026). Von Wikipedia:

[https://de.wikipedia.org/wiki/Normalisierung_\(Datenbank\)](https://de.wikipedia.org/wiki/Normalisierung_(Datenbank)) abgerufen